CSS

CSS - Cascading Style Sheets

- Style `<style> </style>`
  1. Put style in `<style>` tags in the `<head>` element
  2. Put css in separate file, reference file in `<head>` element
- Rules
  - CSS is rule-based.
  - Define selector. `.class` / `#id`
  - `Properties` in curly braces `{}`

```css
p {color: green; font-size: 2em;}           → All <p> elements
h1 {text-transform: uppercase;}             → All <h1> elements
.my-custom-class {padding: 1rem;}           → All instances of class
#my-custom-id {margin: 1rem;}               → All instances of id
p.my-2nd-class {border: 1px solid black;}   → All instances of p-class
```

- Inline

```css
<style>
.large {
font-size: 5em;
}

.bold {
font-weight: 700;
}
</style>

<h2 class="large bold">
I am large and bold.
</h2>
```

- Separate file

```css
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Original title</title>
<link rel="stylesheet" type="text/css" href="styles.css"> (← CSS)
</head>

<body>
<p class="brag">
Using styles.css file referred to in the <head> element.
</p>
</body>
</html>
```

```css
/*styles.css*/
.brag {
color: red;
font-weight: 700;
}
```

- Colors
  - Keywords `white`, `black`, `transparent` → Results in color of keyword
  - RGB *Red, Green, Blue color scale*
    - `255 * 255 * 255 = 16 777 216`
    - `#RRGGBB` Hex: 0-9, A-F
    - `#RRGGBBAA` AA = Alpha 0-100% Opacity
  - HSL *Hue, Saturation, Lightness color scale*
    - 0-360: Angle of color circle
    - 0-100%: Color & Lightness

```css
/*Keywords*/
h2 {color: green;}                          → Green text
.warning {background-color: lightgray;}     → Light Grey background
/*RBG*/
h1 {color: rgb(0-255, 0-255, 0-255);}       → Define color by decimal
.dark-alpha {color: rgba(74, 87, 104, 75%);}→ Define color & Opacity
.dark {color: #4A5768;}                     → Dark Grey
.dark-alpha {color: #4A5768C0;}             → Dark Grey 75% Opacity
/*HSL*/
h1 {color: hsl(120, 50%, 100%);}            → HSL color
h1 {color: hsla(120, 50%, 100%, 75%);}      → HSL color + opacity%
h1 {color: hsla(60, 75%, 50%, 0.9);}        → HSL color + opacityDec
```

- Length
  - Pixels - *Absolute* length unit using monitor's pixels as base
  - `width: 250px; height: 250px;`
    - CSS pixels != device pixels. Represents a unit angle of view.
      - 1 px in CSS not equal to exactly 1px on screen, but somewhat close.
  - Percentage of parent - *Relative* length unit using parent's element length as base
    - `width: 75%; height: 50%;`
  - em *Relative* to current element's font size
    - Example: Inherits 16px from parent element:
      - `font-size: 1.0em; == 16px
      - `padding: 2.0em; == (16x x 2 = 32px)`
  - rem *Relative* to *root* element's font size (usually `<html>` element)
    - `font-size = 1.5 rem == (1.5 x 16px = 24px)`
  - Percentage of viewport. vh/vm *Relative* to current device's viewport width and height.
    - Width: `100vw` translates to `100%`
    - Height: `100vh` translates to `100%`
- Properties
  - Text color
    - `h1 {color: color-value;}`
  - Background color
    - `body {color: color-value; background-color: color-value;}`
  - Border
    - `.all-style {border: solid;}`

      .all-style

- `.all-style-color {border: dotted red;}`

  .all-style-color

- `.all-width-style {border: 1em solid;}`

  .all-width-style

- `.all-width-style-color {border: 1em double #d32f2f;}`

  .all-width-style-color

- `border-radius: 1em 1em 1em 1em;`

  .rounded

- Padding - Create space *between* the content of an *element* and it's border

```css
.padded-box {
padding: 1em; /* equivalent */
padding: 1em 1em; /* equivalent */
padding: 1em 1em 1em 1em; /* equivalent */
border: 1px solid black;
}

.padded-box {
padding-top: 1em;
padding-right: 1em;
padding-bottom: 1em;
padding-left: 1em;
border: 1px solid black;
}
```

- .padded-box

- Margin - Create space *around* an *element's* border

```css
.lonely-box {
margin: 1em; /* equivalent */
margin: 1em 1em; /* equivalent */
margin: 1em 1em 1em 1em; /* equivalent */
border: 1px solid black;
}

.lonely-box {
margin-top: 1em;
margin-right: 1em;
margin-bottom: 1em;
margin-left: 1em;
border: 1px solid black;
}
```

The div below me has 1em margin on all sides. That's why I am so far away.

.lonely-box

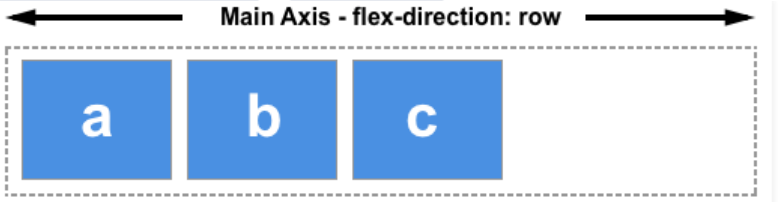
- *The div above me has 1em margin on all sides. That's why I am so far away.*
  - Padding vs Margin

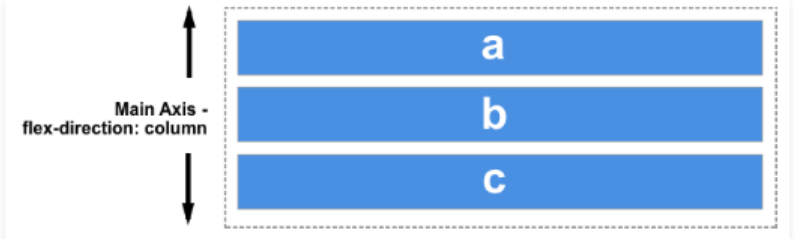
  - Font Size
    - `font-size: 2.5rem` (2 x root element's size, usually 16 if not specified.
- Position - Override default document flow.
  - Static - *Default* position behavior.
  - `position: static;`
  - Relative - Once element is *initially* placed in a position, we can modify it's final position.
  - `position: relative`
  - `top/bottom value;`
  - `left/right value;`
  - Absolute - Will position itself *relative* to the nearest positioned *ancestor*. We can modify it's final position. If no ancestor elements, displayed outside `<html>` element, and relative to the viewport.
  - `position: relative`
  - `top/bottom value;`
  - `left/right value;`
  - Fixed - Will position itself *relative* to the visible part of the viewport. Regardless of scroll position.
  - `position: fixed;`
  - `top/bottom value;`
  - `left/right value;`
  - Sticky - Positions element *relatively* until threshold (e.g scrolling) is met. Then becomes fixed.
  - `position: sticky;`
  - `top/bottom value;`
  - `left/right value;`

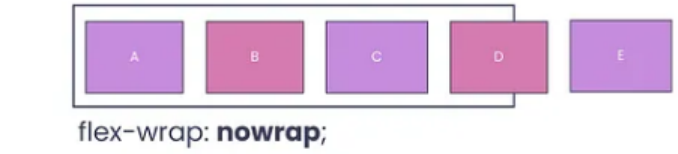
- Flexbox - Flexible box layout `display: flex;`

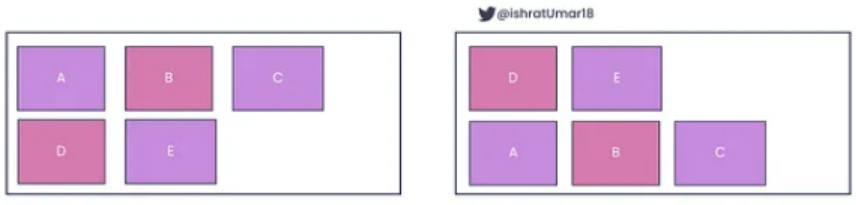
```css
<div class="wrapper">
  <div class="box red"></div>
  <div class="box green"></div>
  <div class="box blue"></div>
</div>

.wrapper {
  display: flex;
  flex-direction: row;
}
```

- 
  - Axes & flex direction
    - `flex-direction: row` / `row-reverse`
      - 

      - `flex-direction: column;` / `column-revese`
      - 

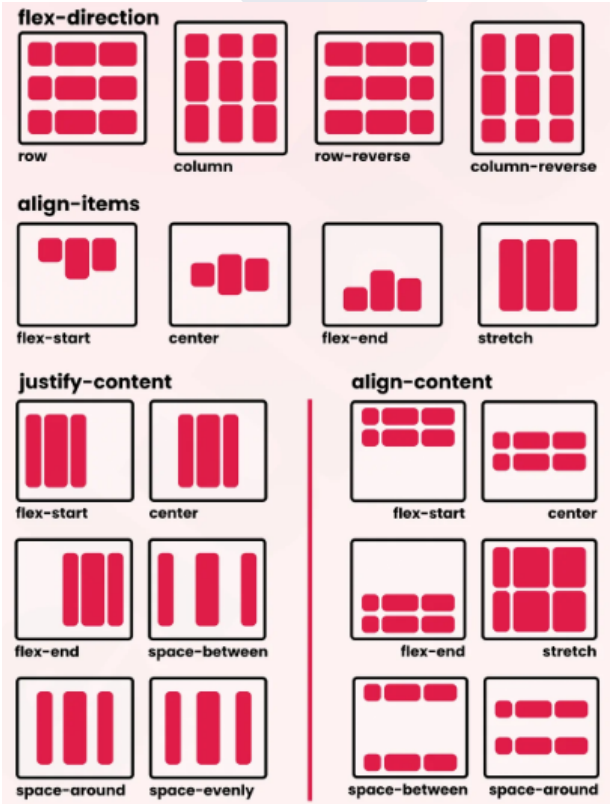
      - Defaults to `row`
      - *Also affected by browsers writing mode. Left-right, or right-left.*
  - Flex wrap - *Wrap* elements over multiple lines
    - 

flex-wrap: **nowrap**;
    - 

flex-wrap: **wrap**; flex-wrap: **wrap-reverse**;
  - Flex basis - The *starting size* of a flex item *along* the container's main axis.
    - `flexbasis: value (eg 100px or auto)`
    - Overrides `width` / `height` when flexbox is in play
    - Commonly set to the smallest acceptible size of the element.
    - With `flex-wrap: wrap;` and `flex-basis: 100%`, each item spans a full line → stacked layout (useful for responsive/mobile design).
  - Flex grow - How much an element is *allowed* to grow in the main axis compared to the other sibling elements.
    - `flex-grow: 0` → Element now allowed to grow in size.

    - If all elements have same grow → All treated equally
- Flex shrink - How much an element is *allowed* to shrink in the main axis compared to the other sibling elements.
- Flex shorthand - `flex` combines `flex-grow`, `flex-shrink`, `flex-basis`

```css
/* unitless */
selector {
flex: 1;
}
/* equivalent */
selector {
flex-grow: 1;
flex-shrink: 1;
flex-basis: 0;
}
```

-
    - Main axis alignment: `justify-content`
    - Cross axis alignment: `align-content`
    - 

- Grid - `display: grid` Use *grids* for full page layouts, flex for one-dimensional components

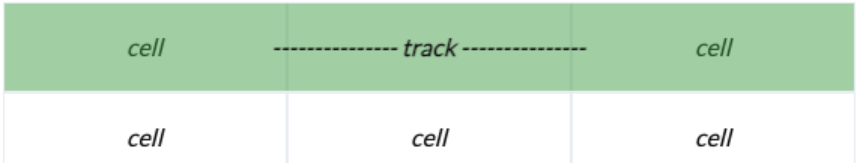
```css
<div class="wrapper">
<div class="box red"></div>
<div class="box green"></div>
<div class="box blue"></div>
</div>

.wrapper {
display: grid;
grid-template-columns: length-unit length-unit ...;
grid-template-rows: length-unit length-unit ...;
}
```

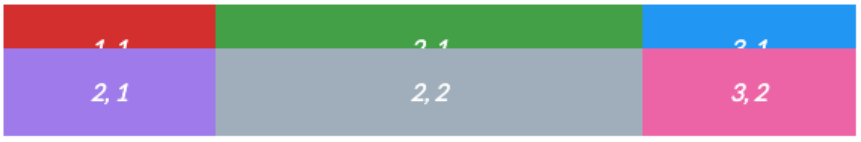
-

- All direct children automatically become grid elements
- Defaults to *row*, and spans the entire width of the container
- Defaults to one *column*.

- Grid tracks - Define *number* and *length* of rows and columns.
  - Build grid with any normal length unit, or `fr` -fractional unit
  - `grid-template-rows: length-unit length-unit ...`
  - `grid-template-columns: length-unit length-unit ...`




```css
.wrapper {
  display: grid;
  grid-template-rows: 25% 50% 25%%;
  grid-template-columns: 25% 50% 25%%;
}
```



- Fractional unit `fr` - a share of leftover space in a CSS grid
  - Total space = container - fixed units.
  - Fractions split that remainder equally.
  - Tracks expand if space allows, shrink to fit content if not.

```css
.wrapper {
display: grid;
grid-template-rows: 1fr 1fr;
}
```

- Minmax - defines a track that won't shrink below *min* and won't grow beyond *max*.
  - `minmax(100px, 1fr)` → row is at least 100px, but can expand up to 1 fraction of free space.
- Repeat - `repeat(count, value)` = shorthand for writing the same grid track multiple times.
  - `repeat(3, 1fr)` → `1fr 1fr 1fr`.
- Grid positioning - place items by specifying start/end grid lines.
  - row/column: `grid-row-start`, `grid-row-end`, `grid-column-start`, `grid-column-end`.
  - Shorthands:
    - `grid-row`, `grid-column` = start / end.
    - `grid-area` = `row-start / col-start / row-end / col-end`.
  - Example:
    -
      - `grid-area: 1 / 2 / 3 / 4;` → rows 1-3, cols 2-4.
  - `span` keyword: span a number of tracks from the start line.

    - `grid-row: 1 / span 2;` → rows 1-3.
- Grid template area `grid-template-areas` - name parts of the grid and place items by name instead of line numbers.
    - Define rows as quoted strings → each word = a cell's area name.
    - Same name across cells = one larger area.

```css
grid-template-areas:
  "header  header"
  "sidebar content"
  "footer  footer";
```

-
    - → creates named regions: `header`, `sidebar`, `content`, `footer`.
    - Items use `grid-area: name;` to slot in.
- Gutters - spacing between grid tracks.
    - `grid-row-gap` = vertical gap, `grid-column-gap` = horizontal gap.
    - Shorthand: `grid-gap: row col;`
        - Example: `grid-gap: 1rem 30px;` → row gap = 1rem, column gap = 30px.
- Responsive design - making sites adapt to different screen sizes.
    - Why: Over half of web traffic = mobile.
- Mobile-first design = start with small screens, then enhance for larger ones via media queries.
    - Follows progressive enhancement: core first → extras later.
    - Forces solving space constraints early.
    - Once mobile works, larger layouts scale more easily.
- Breakpoints = screen-width thresholds used to adapt layouts.
  Common ones:
    - <640px → very small (phones, ~360px logical width).
    - 640px → small (portrait phones).
    - 768px → medium (old laptops, tablets).
    - 1024px → large (landscape tablets, laptops).
    - 1280px+ → extra large (desktops).
    - Use: media queries tweak grid/flex settings (rows, columns, `flex-basis`) at these widths.
- Media queries = conditional CSS rules based on device features (width, height, resolution, etc.).

```css
@media screen and (min-width: 640px) {
  .wrapper { flex-basis: 100%; }
}
```

-
    - Flow: `@media` → tests → styles if tests pass.
    - Allows different layouts/styles per breakpoint or device.
- Pseudo-classes = CSS keywords that style elements based on state or external factors.
    - Syntax: `selector:pseudo-class { property: value; }`
        - `:hover` → mouse over.
        - `:focus` → when element is selected (clicked/tabbed into).
        - `:visited` / `:link` → link history.
    - Let us define element states without adding extra classes in HTML.
- Transitions = smooth animations between CSS property changes.
    - Without transition → instant change.
    - With transition → change happens over time.
    - Not all properties can animate (avoid `auto`).
    - Shorthand parts:
      `transition: property duration timing-function delay;`

1. transition-property: the CSS property to which a transition should be applied

2. transition-duration: time the transition should take to reach the end state
3. transition-timing-fuction: the function responsible for calculating intermediate property values
4. transition-delay: time to wait before starting the transition animation

- Defaults:
  - `property: all`
  - `timing-function: ease`
  - `delay: 0`
  - `duration` = required